eitkit

Electrical Impedance Tomography Reconstruction Toolkit

Abd'gafar Tunde Tiamiyu
School of Mathematics, Jilin University
Changchun 130012, Jilin, China
abdgafartunde@yahoo.com

Version 0.1.1 — July 2026
DOI: 10.5281/zenodo.21319128

Summary

eitkit is an open-source Python package for 2-D difference Electrical Impedance Tomography (EIT). The forward problem is solved by P1 finite elements with a gap electrode model, and the Jacobian is computed by the adjoint method with sparse LU factorisation reuse. Inverse reconstruction employs classical regularisation: Tikhonov (L^2) with L-curve parameter selection and Total Variation (L^1) via ADMM with auto-scaled penalty. Mesh generation uses the DistMesh2D algorithm of Persson & Strang (2004). The toolkit requires only NumPy, SciPy, and Matplotlib. Version 0.1.1 ships with a demo notebook, five synthetic phantom shapes, and 154 unit tests.

1 Overview

Electrical Impedance Tomography (EIT) reconstructs the interior conductivity distribution $\sigma(x)$ of a domain $\Omega \subset \mathbb{R}^2$ from boundary voltage measurements. The mathematical problem, posed by Calderón [1], is severely ill-posed: small perturbations in boundary data can induce arbitrarily large errors in any reconstruction [2].

eitkit implements the complete **difference EIT** pipeline:

Stage	Implementation
Mesh generation	DistMesh2D triangular mesh on the unit disc
Forward problem	P1-FEM with gap electrode model
Jacobian	Adjoint method with sparse LU factorisation reuse
Inverse problem	Tikhonov (L^2) + L-curve and TV (L^1) via ADMM

2 Forward Problem

2.1 Governing Equation

Let $\Omega \subset \mathbb{R}^2$ be a bounded Lipschitz domain. Given a conductivity $\sigma \in L^\infty(\Omega)$ with $\sigma(x) \geq c > 0$, the electric potential $u \in H^1(\Omega)$ satisfies

$$-\nabla \cdot (\sigma \nabla u) = 0 \quad \text{in } \Omega, \tag{1}$$

$$\sigma \frac{\partial u}{\partial n} = j \quad \text{on } \partial\Omega, \tag{2}$$

where j is the applied current density with $\int_{\partial\Omega} j \, ds = 0$. A ground condition $u(x_0) = 0$ at one interior node ensures uniqueness.

2.2 Gap Electrode Model

Each of the L electrodes is modelled as a single boundary node. A stimulation pair (e^+, e^-) injects current $+I$ at electrode e^+ and extracts $-I$ at e^-. The Neumann load vector $f \in \mathbb{R}^N$ has entries

$$f_i = \begin{cases} +I & \text{node } i \text{ is electrode } e^+, \\ -I & \text{node } i \text{ is electrode } e^-, \\ 0 & \text{otherwise.} \end{cases}$$

2.3 P1 Finite Element Discretisation

The domain is triangulated with M linear elements on N nodes. Writing $u = \sum_{i=1}^{N} u_i \varphi_i$ with P1 hat functions φ_i, the weak form yields

$$K(\sigma)\, u = f, \tag{3}$$

where the global stiffness matrix is assembled element-wise:

$$K = \sum_{e=1}^{M} K^e, \qquad K^e_{ij} = \frac{\sigma_e}{4A_e}(b_i b_j + c_i c_j), \quad i, j \in \{0, 1, 2\}.$$

The gradient coefficients are $b_i = y_{i+1} - y_{i+2}$, $c_i = x_{i+2} - x_{i+1}$ (indices mod 3), and A_e is the element area. Assembly is vectorised over elements using NumPy advanced indexing, producing a `csr_array` for efficient sparse arithmetic.

2.4 Measurement Protocol

The standard **adjacent drive / adjacent measurement** protocol is used. For L electrodes there are L drive steps; step k sources current at electrode k and sinks at $(k+1) \bmod L$. Per drive step, $L - 3$ voltage measurements are taken across adjacent electrode pairs $(k+2, k+3), \ldots, (k+L-2, k+L-1)$ (all indices modulo L). The total number of measurements is $P = L(L-3)$; for $L = 16$ this gives $P = 208$.

2.5 Difference EIT and the Jacobian

Absolute voltages are difficult to calibrate experimentally. Difference EIT measures voltage changes relative to a reference state σ_0: $\delta V = V(\sigma) - V(\sigma_0)$. Linearising around σ_0 gives the Born approximation

$$\delta V \approx J(\sigma_0)\, \delta\sigma, \tag{4}$$

where $J \in \mathbb{R}^{P \times M}$ is the Jacobian and $\delta\sigma = \sigma - \sigma_0 \in \mathbb{R}^M$ is the conductivity perturbation.

The Jacobian is computed by the **adjoint method**. For measurement i associated with drive step k and electrode pair (e^+, e^-),

$$J_{ie} = -\int_{\Omega_e} \nabla u_k \cdot \nabla \phi_i \, d\Omega, \tag{5}$$

where u_k is the forward potential for drive step k and ϕ_i is the adjoint potential (unit current injected at e^+, extracted at e^-). For the adjacent protocol, at most L adjoint solves are needed, and the factorised stiffness matrix is reused for all solves via `scipy.sparse.linalg.splu`.

3 Inverse Problem

3.1 Tikhonov Regularisation (L^2)

The Tikhonov solution minimises

$$\delta\hat{\sigma} = \arg\min_{\delta\sigma} \left\{ \tfrac{1}{2}\|J\delta\sigma - \delta V\|_2^2 + \lambda\|\delta\sigma\|_2^2 \right\}, \tag{6}$$

with closed-form solution $\delta\hat{\sigma} = (J^TJ + \lambda I)^{-1}J^T\delta V$. The regularisation parameter λ is chosen automatically via the L-curve method [4]: the optimal λ corresponds to the point of maximum curvature on the log-log plot of residual norm against solution norm.

3.2 Total Variation (L^1 / ADMM)

TV regularisation promotes piecewise-constant reconstructions:

$$\delta\hat{\sigma} = \arg\min_{\delta\sigma} \left\{ \tfrac{1}{2}\|J\delta\sigma - \delta V\|_2^2 + \alpha\|D\delta\sigma\|_1 \right\}, \tag{7}$$

where $D \in \mathbb{R}^{F\times M}$ is a sparse finite-difference operator on element-to-element edges. The problem is solved by the Alternating Direction Method of Multipliers (ADMM) [5] with splitting $z = D\delta\sigma$. The σ-update is a quadratic system solved by direct factorisation, the z-update is element-wise soft-thresholding, and the penalty parameter ρ is auto-scaled as $\rho = \mathrm{tr}(J^TJ)/\mathrm{tr}(D^TD)$.

4 Package Structure and Availability

Module	Contents
`eitkit.mesh`	DistMesh2D mesh generation, electrode placement
`eitkit.forward`	P1 FEM assembly, gap electrode model, adjoint Jacobian
`eitkit.protocol`	Adjacent drive and measurement patterns, AWGN noise
`eitkit.inverse`	Tikhonov (L^2) + L-curve, TV-ADMM (L^1)
`eitkit.utils`	Five phantom shapes, publication-quality visualisation

Installation from source:

```
git clone https://github.com/abdgafartunde/eit-reconstruction-toolkit
cd eit-reconstruction-toolkit
pip install -e ".[dev]"
```

The core dependency stack is NumPy ≥ 1.24, SciPy ≥ 1.10, and Matplotlib ≥ 3.7. Optional extras include PyTorch and JAX for future learning-based extensions. A complete interactive example is provided in `examples/01_reconstruction_demo.ipynb`.

5 References

References

[1] A.P. Calderón. On an inverse boundary value problem. In *Seminar on Numerical Analysis and its Applications to Continuum Physics*, Rio de Janeiro, 1980.

[2] N. Mandache. Exponential instability in an inverse problem for the Schrödinger equation. *Inverse Problems*, 17(5):1435–1444, 2001.

[3] P.-O. Persson and G. Strang. A simple mesh generator in MATLAB. *SIAM Review*, 46(2):329–345, 2004.

[4] P.C. Hansen. Analysis of discrete ill-posed problems by means of the L-curve. *SIAM Review*, 34(4):561–580, 1992.

[5] S. Boyd, N. Parikh, E. Chu, B. Peleato, and J. Eckstein. Distributed optimization and statistical learning via the alternating direction method of multipliers. *Foundations and Trends in Machine Learning*, 3(1):1–122, 2011.

[6] A. Adler and W.R.B. Lionheart. Uses and abuses of EIDORS: an extensible software base for EIT. *Physiological Measurement*, 27(5):S25–S42, 2006.